Exercício

- Faça um método que receba um *array* de inteiros e um número inteiro *x* e retorne um valor booleano informando se o elemento *x* está contido no *array*

```
public boolean verifica_bool(int vet[],int x){
  for(int i=0;i<vet.length;i++){
    if(vet[i]==x){
      return true;
    }
  }
  return false
}
```

Exercício

- Repita o exercício acima considerando que os elementos do *array* estão ordenados de forma crescente. Uma sugestão seria começar a pesquisa pelo meio do *array*

```
public boolean mostaExistencia(int vet[], int b) {

    int a=0;

    if (vet[vet.length / 2] >= b) {

      a = vet.length / 2;

    }

    for (int i = a; i < vet.length; i++) {

      if (vet[i] == b)

        return true;

    }
    return false;

  }
```

Exercício

- Faça um método que receba um *array* de inteiros e mostre na tela o maior e o menor elementos do *array*.

```
public void MaiorEMenor(int v[], int x) {

  int maior = vet[0];

  int menor = vet[0];


  for (int i = 0; i < v.length; i++) {

    if (vet[i] > maior)

      maior = vet[i];

    if (vet[i] < menor)

      menor = vet[i];

  }
  System.out.println("Maior: " + maior);

  System.out.println("Menor: " + menor);

}
```

Exercício

- Repita o exercício acima fazendo menos comparações com os elementos do *array*

```
public void MaiorEMenor(int vet[], int x) {
  int maior = v[0];
  int menor = v[0];
  for (int i = 0; i < v.length; i++) {

    if (vet[i] > maior)

      maior = v[i];

    else if (vet[i] < menor)
```

```
            menor = vet[i];
        }
        System.out.println("Maior: " + maior);
        System.out.println("Menor: " + menor);
    }
```

Exercício

- Qual das duas versões é mais fácil de entender?

```
boolean isConsoante(String s, int i){
        boolean resp= true;

        if (i == s.length()){
                resp = true;
        } else if (isConsoante(s.charAt(i)) == false){
                resp = false;
        } else {
                resp = isConsoante(s, i + 1);
        }

        return resp;
}
```

```
boolean isConsoante(String s, int i){
        boolean resp= true;

        if (i < s.length()){
                if (!isConsoante(s.charAt(i))){
                        resp = false;
                } else {
                        resp = isConsoante(s, i + 1);
                }
        } else {
                resp = true;
        }

        return resp;
}
```

O código da esquerda.

Exercício

- Qual é a sua opinião sobre o código **REAL** abaixo?

```
Unidade recuperarUnidadeComCodigoDeUCI(Unidade unidadeFilha) {
        Unidade retorno = null;

        if (unidadeFilha.getCodUci() != null && !unidadeFilha.getCodUci().isEmpty()) {
                retorno = unidadeFilha;
        } else {
                retorno = unidadeFilha.getUnidadeSuperior();
        }

        while (retorno == null || retorno.getCodUci() == null || retorno.getCodUci().isEmpty()) {
                retorno = retorno.getUnidadeSuperior();
        }

        return retorno;
}
```

Além do o custo do programa ser bem elevado, o código também, apresenta condições desnecessárias ficando mais complexo o seu entendimento.

Exercício

- Qual é a diferença entre os dois métodos abaixo?

```
int m1(int i){
      return i--;
}


int m2(int i){
      return --i;
}
```

O primeiro código retorna o valor que foi recebido de i para depois realizar a subtração fazendo o valor final retornado subtrair por -1. Já o segundo subtrai 1 de i antes de retornar o valor, fazendo com que o retorno do método seja i - 1;